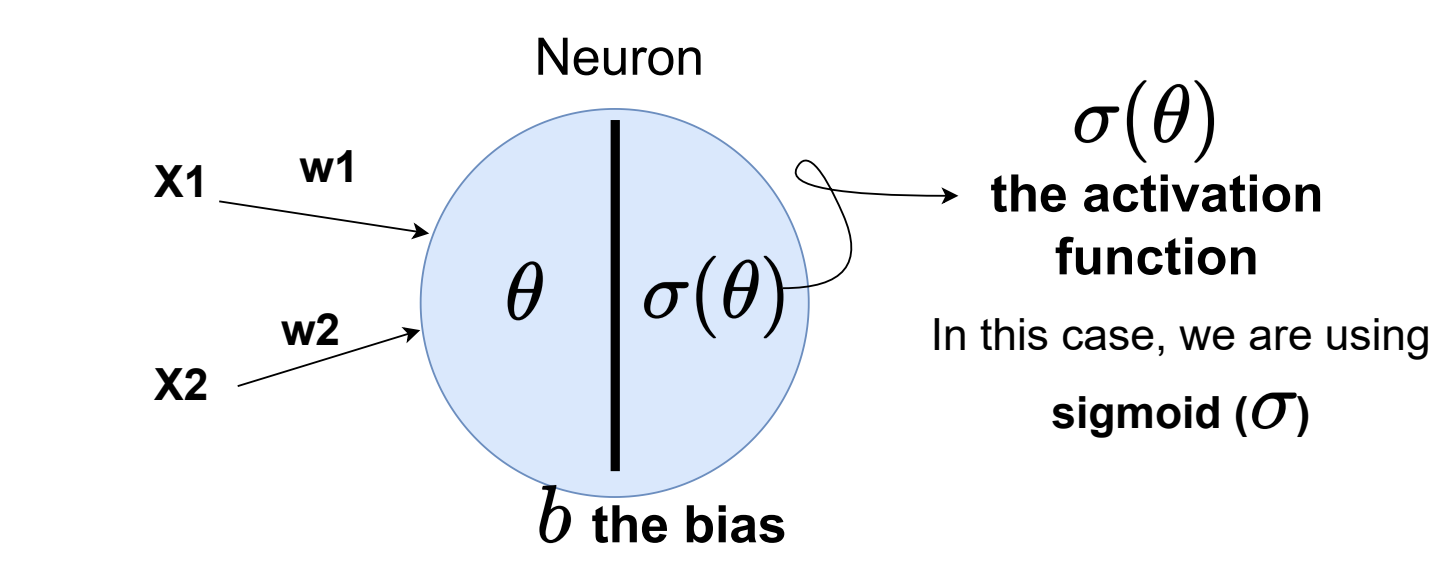


Feed Forward Neural Network

Example; how to calculate the input and the output in Feed Forward Network.
Similar to the example in Slide 8

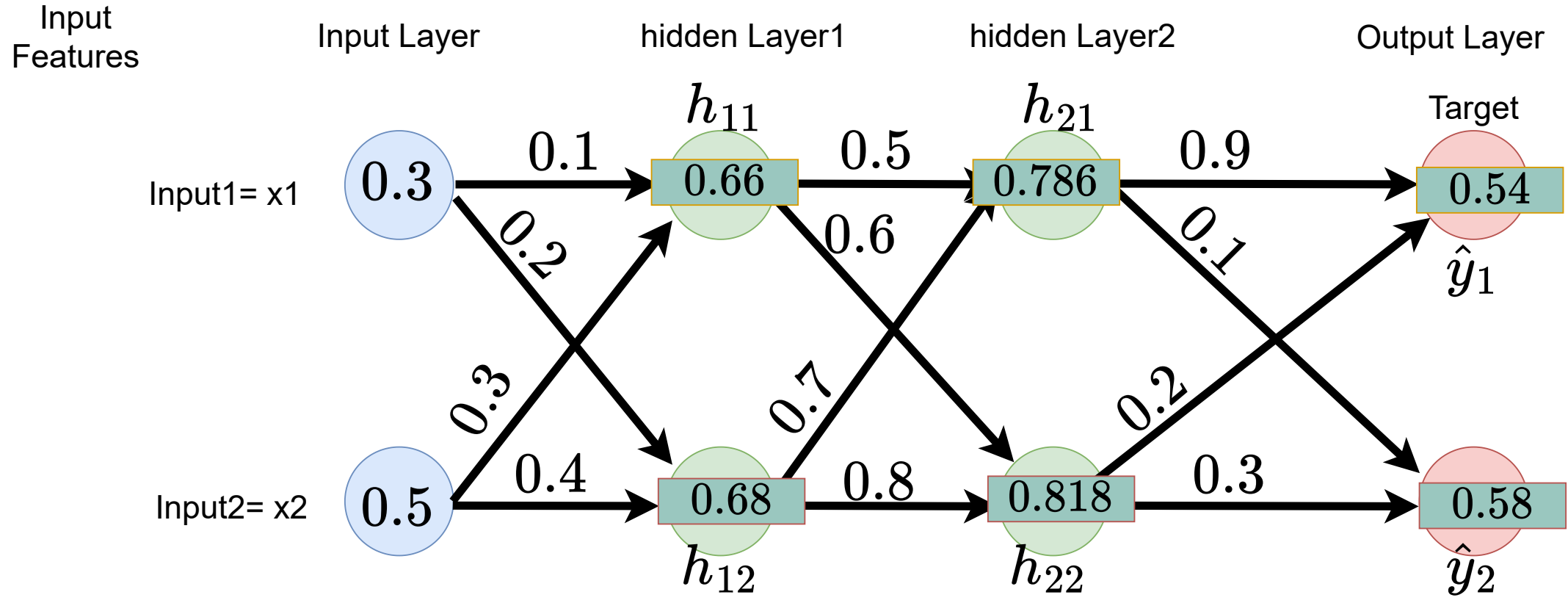
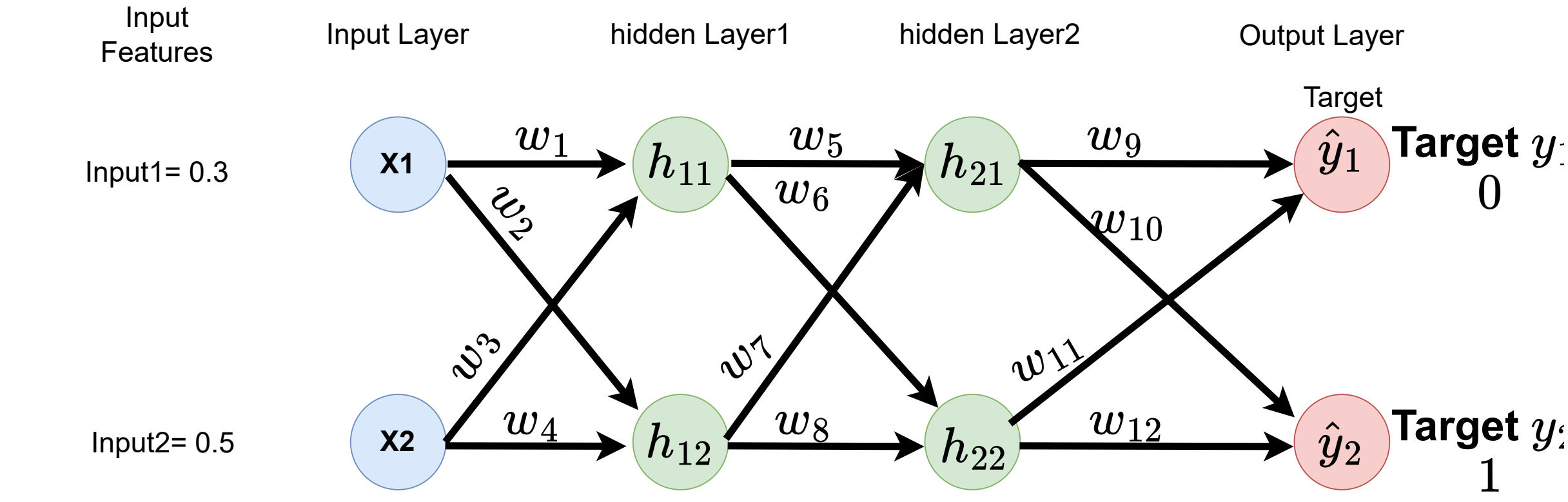


You should compute the input for each neuron as follows

$$\theta = x_1w_1 + x_2w_2 + b$$

if there is any bias

The activation function is used to compute the output of the neuron

$$\sigma(\theta) = \frac{1}{1+e^{-\theta}} = \frac{1}{1+e^{-x_1w_1+x_2w_2+b}}$$


Given, the input x1 and x2
The weights are randomly initialized for the first round (epoch) (All of them)
The bias = 0.5 is the same (randomly initialized)

$$\begin{aligned} h_{11} &= x_1w_1 + x_2w_3 + b \\ &= 0.3 * 0.1 + 0.5 * 0.3 + 0.5 \\ &= 0.03 + 0.15 + 0.5 \\ &= 0.68 \end{aligned}$$
$$\begin{aligned} h_{12} &= x_1w_1 + x_2w_4 + b \\ &= 0.3 * 0.2 + 0.5 * 0.4 + 0.5 \\ &= 0.06 + 0.2 + 0.5 \\ &= 0.76 \end{aligned}$$

$$\sigma(h_{11}) = \frac{1}{1+e^{-0.68}} = 0.66$$
$$\sigma(h_{12}) = \frac{1}{1+e^{-0.76}} = 0.68$$

These values will be used as input for the next layer

$$\begin{aligned} h_{21} &= \sigma(h_{11})w_5 + \sigma(h_{12})w_7 + b \\ &= 0.66 * 0.5 + 0.68 * 0.7 + 0.5 \\ &= 0.33 + 0.476 + 0.5 \\ &= 1.306 \end{aligned}$$
$$\begin{aligned} h_{22} &= \sigma(h_{11})w_6 + \sigma(h_{12})w_8 + b \\ &= 0.66 * 0.6 + 0.76 * 0.8 + 0.5 \\ &= 1.504 \end{aligned}$$

$$\sigma(h_{21}) = \frac{1}{1+e^{-1.306}} = 0.786$$
$$\sigma(h_{22}) = \frac{1}{1+e^{-1.504}} = 0.818$$

These values will be used as input for the next layer. In this case, it will be the output layer

$$\begin{aligned} \hat{y}_1 &= \sigma(h_{21})w_9 + \sigma(h_{22})w_{11} \\ &= 0.786 * 0.9 + 0.818 * 0.2 \\ &= 0.1636 \end{aligned}$$
$$\begin{aligned} \hat{y}_2 &= \sigma(h_{21})w_{10} + \sigma(h_{22})w_{12} \\ &= 0.786 * 0.1 + 0.818 * 0.3 \\ &= 0.324 \end{aligned}$$

$$\sigma(\hat{y}_1) = 0.54$$
$$\sigma(\hat{y}_2) = 0.58$$

$$\begin{aligned} Error/L &= \text{Mean Squared Error} \\ &= \frac{1}{2} [(\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2] \\ &= \frac{1}{2} [(0.54 - 0)^2 + (0.58 - 1)^2] \\ &= \frac{1}{2} (0.2916 + 0.1764) \\ &= \frac{0.468}{2} \\ &= 0.234 \end{aligned}$$

At this point, all of these calculations are only for the first epoch.
We also expect that the error should be minimal to get the optimal weights.

At this stage, you can not change your input, the neuron nodes, or the target. However, you can update the weights.

Therefore, here comes the role of backpropagation, which uses the learning rate to recalculate these weights and update them for the next round (the next epochs).